LLM-Based Triage of GPU Numerical Failures from Public Issue Reports

Paper #XXX
Anonymous Submission
Anonymous

Abstract

GPU software stacks increasingly combine compiler transformations, generated kernels, mixed precision, and high-level Python interfaces. Their failure reports are difficult to triage because symptoms such as NaNs, wrong answers, overflow, dtype errors, compiler crashes, and performance regressions often appear in the same public issue threads. This paper presents GPUTriage, a human-labeled benchmark and LLM-based triage system for GPU numerical-failure reports mined from public repositories. We separate weak retrieval labels from human gold labels and build a 1,191-issue benchmark over eight GPU-related software projects with a seven-class taxonomy. Weak keyword-derived labels reach only 50.5% accuracy, showing that issue-search labels are not reliable ground truth. A fine-tuned FLAN-T5-base triage model reaches 80.5% held-out accuracy and 0.778 macro F1, while an LLM/deterministic agreement mode reaches 89.3% accuracy at 68.3% coverage. Leave-one-repository-out evaluation drops to 66.8%, exposing cross-project transfer as the central systems challenge. We provide an anonymized artifact package with labels, audit logs, evaluation tables, model outputs, and root-cause evidence files for review.

1 Introduction

GPU systems failures are no longer isolated to hand-written CUDA kernels. A modern AI or scientific workload can pass through tensor libraries, graph rewriters, JIT compilers, code generators, mixed-precision policies, and vendor-specific execution backends before reaching hardware. When the result is wrong, slow, non-finite, or impossible to compile, the first structured record is often a public issue report. These reports are valuable systems data, but they are noisy: a thread mentioning "NaN" may be a true invalid-value bug, a missing-data question, a tolerance dispute, or a performance benchmark; a "dtype" issue may be a casting bug, unsupported API request, crash, or precision error.

This paper asks whether public issue reports can support reliable triage of GPU numerical failures. The task is practical for systems builders: maintainers need to route reports to compiler, runtime, dtype, numerical-accuracy, or performance experts; researchers need reusable traces of reliability problems; and automated debugging tools need labeled reports that distinguish symptoms from unrelated noise. Existing public issue trackers contain the raw material, but weak labels from search queries are not enough.

We present GPUTriage, an LLM-based triage system and benchmark for categorizing GPU numerical-failure reports. GPUTriage mines public GPU software issue trackers, repairs weak labels through human annotation, and evaluates both full-coverage and selective triage. The benchmark contains 1,191 human-labeled issues from JAX, PyTorch, RAPIDS cuML, Triton, RAPIDS cuDF, CuPy, Numba, and Apache TVM. Each issue receives one primary label from a seven-class taxonomy: invalid values, overflow/underflow, precision/tolerance, dtype/casting, crash/compile, performance-only, or not a numerical failure.

The contributions are:

- We show that weak issue-search labels are insufficient for GPU numerical-failure triage, reaching only 50.5% accuracy on the final benchmark.
- We construct a 1,191-issue human-labeled benchmark with audit logs, taxonomy repairs, a 250-row evidence-coded root-cause extension, and reproducibility artifacts.
- We implement and evaluate an LLM-based triage model that reaches 80.5% held-out accuracy and a selective agreement mode that reaches 89.3% accuracy at 68.3% coverage.
- We evaluate chronological and leave-one-repository-out splits, identifying cross-project transfer as the main remaining barrier.

We organize the evaluation around four systems questions: **RQ1**: Are weak labels from issue search usable as ground truth? **RQ2**: Does a task-specific LLM improve report triage over transparent text baselines? **RQ3**: Can the system trade coverage for higher precision when reports are ambiguous? **RQ4**: Does triage transfer across GPU software ecosystems, or does repository-specific language dominate?

2 Problem and Dataset

2.1 Task

Given a public issue report from a GPU software project, the triage task is to predict the dominant failure category. The input includes title, body, repository metadata, and available public issue context. The output is exactly one primary label. This paper does not claim to prove root cause from

Table 1. Human-labeled benchmark distribution.

Primary label	Issues
precision_tolerance	368
dtype_casting	193
crash_compile	192
not_numerical_failure	147
nan_inf	143
overflow_underflow	94
performance_only	54
Total	1191

Table 2. Triage system stages.

Stage	Role
Mining	Retrieve public reports with numerical, dtype, compiler, and wrong-answer terms.
Repair	Normalize weak and human labels into the seven-class taxonomy while logging every repair.
Training	Train deterministic text baselines and a fine-tuned FLAN-T5-base label generator.
Deployment	Emit a label, confidence mode, or abstention for human triage.

text alone. Instead, it classifies the report that a maintainer or triage assistant would see before full debugging.

2.2 Mining Public Reports

We began with a silver seed mined from GPU-related repositories using terms such as `nan`, `inf`, `overflow`, `dtype`, `precision`, `tolerance`, `incorrect`, and `wrong result`. The retrieval query and keyword heuristics produced a candidate label for each issue, but those labels were treated as weak supervision rather than ground truth. We then constructed a pilot gold set and expanded it with 1,000 additional human-labeled records. The final benchmark contains 1,191 issues.

The final benchmark is deliberately repository-diverse rather than dominated by one project. It includes compiler-centered reports, tensor-library reports, Python/JIT reports, and GPU data-frame or array-library reports. That diversity matters because the same surface symptom can mean different things across systems. A dtype report in an array library often concerns API compatibility or casting semantics; a dtype report in a compiler stack may indicate lowering, legalization, or backend code generation. The benchmark therefore tests both within-distribution triage and transfer across software ecosystems.

2.3 Taxonomy

The taxonomy separates the observed report category from deeper root cause. `nan_inf` covers non-finite outputs; `overflow_underflow` covers numerical range failures; `precision_tolerance` covers wrong-answer and tolerance failures; `dtype_casting` covers dtype promotion, unsupported dtype, and casting semantics; `crash_compile` covers compiler/runtime exceptions and illegal memory failures; `performance_only` covers speed or memory complaints without a correctness failure; and `not_numerical_failure` covers feature requests, documentation issues, API questions, and unrelated reports.

2.4 Human Label Quality

The expanded annotation file included labels outside the final taxonomy, including API feature requests, generic needs-review labels, and compiler/code generation descriptions. We ran a deterministic taxonomy-normalization pass and preserved the full repair log. In total, 180 rows were mapped into the final taxonomy without silently dropping records. A blind audit over 119 rows produced 80.7% agreement and Cohen's $\kappa = 0.721$ between two annotators. A third adjudication pass and a second 300-row adjudication pass focused on low-confidence and model-error examples. Together, the passes applied 419 human updates and changed 211 primary labels.

The review process also exposed weaknesses in the label schema. Early labels mixed symptoms, suspected causes, and issue-management categories. We corrected that by requiring a single primary report label and moving deeper cause evidence into separate fields. For a maintainer-facing queue, this is the right separation: first decide whether the report is numerical correctness, dtype, compiler/runtime, performance, or non-numerical support; then ask whether later evidence identifies a root cause.

3 Triage System

GPUTriage has four stages. First, the ingestion layer normalizes public issue metadata and text into benchmark rows. Second, the taxonomy layer maps weak and human labels into the final seven-class label space while preserving repair provenance. Third, deterministic baselines train on TF-IDF, BM25, naive Bayes, linear SVM, and logistic regression features. Fourth, the LLM stage fine-tunes a FLAN-T5-base sequence-to-sequence classifier to emit one taxonomy label from issue text.

The primary LLM split contains 945 training rows, 123 validation rows, and 123 held-out test rows. To reduce class imbalance, the training set is balanced by oversampling minority classes to 2,058 examples. The selected checkpoint starts

from a prior expanded-gold checkpoint and is retrained on the v2 canonical labels. We also evaluate selective modes. A deterministic voting ensemble can abstain unless at least k voters agree. The LLM/deterministic agreement mode answers only when the fine-tuned LLM and deterministic ensemble predict the same label.

This keeps the LLM role narrow. The model does not synthesize new reports, rewrite evidence, or infer private information. It maps observed issue text into an auditable taxonomy and can be compared directly with retrieval and linear baselines. The deterministic ensemble remains useful because it provides a transparent fallback and a disagreement signal for selective prediction.

3.1 Implementation and Artifact

The implementation is intentionally simple enough to be reproducible. The data pipeline stores mined issue records, canonical labels, audit files, and taxonomy-repair logs as CSV and JSONL files. Deterministic baselines use standard sparse text features and are rerunnable from the released scripts. The LLM training packet is emitted as sequence-to-sequence examples whose targets are exactly the seven taxonomy labels. Evaluation scripts regenerate the reported tables, per-class summaries, abstention metrics, chronological split, and leave-one-repository-out split.

The artifact is part of the contribution rather than only a supplement. It contains the benchmark, completed blind-audit files, adjudication outputs, model predictions, error appendix, and root-cause evidence extension. During review, public repository and archival links are removed from the paper to preserve anonymity; the camera-ready version can restore those links after acceptance.

4 Evaluation

4.1 Protocol

We report accuracy and macro F1. Macro F1 is needed because the benchmark is imbalanced: precision/tolerance reports are the largest class, while performance-only reports are the smallest. We evaluate four settings: full-coverage shuffled evaluation, chronological evaluation on later issues, leave-one-repository-out transfer, and selective prediction with abstention.

4.2 Full-Coverage Results

Table 3 shows that weak candidate labels reach only 50.5% accuracy, confirming that raw issue-search labels are not reliable supervision. Linear text models substantially improve with human labels, and a voting ensemble reaches 73.8% accuracy and 0.712 macro F1. The fine-tuned LLM reaches 80.5% held-out accuracy and 0.778 macro F1 on its test split, outperforming deterministic baselines on the same held-out rows. A local zero-shot Llama 3.2 3B comparison reaches only 31.7% accuracy and 0.322 macro F1 on those rows.

Table 3. Full-coverage triage results.

Model	Accuracy	Macro F1
Majority baseline	0.309	0.067
Candidate weak label	0.505	0.426
BM25 nearest neighbor	0.571	0.534
TF-IDF logistic regression	0.715	0.688
TF-IDF linear SVM	0.732	0.703
Voting ensemble	0.738	0.712
FLAN-T5-base LLM	**0.805**	**0.778**

Table 4. Generalization beyond shuffled evaluation.

Setting / model	Accuracy	Macro F1
Chronological, linear SVM	0.732	0.697
Chronological, bigram logistic	**0.766**	**0.738**
Chronological, voting ensemble	0.745	0.722
Leave-one-repo, linear SVM	**0.668**	**0.648**
Leave-one-repo, voting ensemble	**0.668**	0.647

Table 5. Weakest leave-one-repository-out transfer cases.

Held-out repo	Issues	Ensemble acc.
Numba	65	0.462
CuPy	82	0.439
RAPIDS cuDF	132	0.545

The weak-label result answers RQ1: keyword retrieval is useful for candidate collection but not for supervision. The 50.5% weak-label accuracy is only better than the majority baseline because issue-search terms are correlated with failure language; it still mislabels many reports whose text contains generic stack traces, dtype names, or numerical terms without a numerical failure. The LLM result answers RQ2: the task-specific model improves over transparent deterministic baselines, but the deterministic models remain useful for debugging and confidence estimation.

4.3 Generalization and Selective Triage

Chronological evaluation trains on older issues and tests on newer issues. The best model in this setting, bigram TF-IDF logistic regression, reaches 76.6% accuracy and 0.738 macro F1, while the voting ensemble reaches 74.5% accuracy. Leave-one-repository-out evaluation is harder: the best deterministic models reach 66.8% accuracy. CuPy and Numba are the weakest held-out repositories because dtype/API compatibility, CUDA runtime failures, and compiler language blur the boundary between numerical and non-numerical categories.

Table 6. Deterministic ensemble ablation.

Mode	Accuracy	Macro F1
Full deterministic ensemble	0.768	0.753
No weak candidate label	0.761	0.748
Linear-model vote only	0.757	0.745
Candidate label + SVM	0.765	0.751

Table 7. Selective prediction with abstention.

Rule	Cov.	Acc.	F1
Vote ≥ 3	0.926	0.769	0.744
Vote ≥ 4	0.735	0.842	0.803
Vote ≥ 5	0.301	0.939	0.733
LLM/ensemble agree	0.683	0.893	0.844

This is the most important negative result. Shuffled and chronological splits show that the benchmark is learnable, but holding out a whole repository exposes vocabulary and workflow shift. Numba issues often mix Python exceptions, LLVM compilation, CUDA behavior, and user-support language. CuPy issues often mix dtype compatibility with numerical symptoms. These are exactly the cases where a systems triage assistant should surface uncertainty rather than force a confident label.

Table 6 shows that the weak candidate label is helpful as a feature-like signal but not as truth. Removing it lowers the deterministic ensemble only slightly, while using it alone is far worse. This supports the design choice to preserve weak labels for analysis but base the benchmark on human annotation.

4.4 Error Analysis

The dominant errors are boundary cases that systems maintainers also find hard. The largest confusion is `dtype_casting` predicted as `precision_tolerance`; many dtype failures appear first as numerical mismatches. `nan_inf` reports are often predicted as `precision_tolerance` when the issue is written around failed accuracy checks. `crash_compile` reports are confused with numerical labels when the exception occurs inside a failed computation. These patterns motivate multi-label secondary-cause prediction and deeper root-cause linking to fixes.

The qualitative appendix contains concrete issue snippets for these boundary cases. These examples are important for ATC because they show that the benchmark is not an artificial label-matching exercise. Reports often contain enough evidence to route the issue but not enough evidence to prove root cause, and the hard cases are exactly where maintainers would want a tool to defer.

Table 8. Root-cause extension label counts.

Evidence label	Rows
Numerical algorithm / tolerance	74
Non-bug, feature, docs, or support	69
Dtype or type semantics	50
Compiler backend or runtime	30
Performance or resource behavior	27
Total	250

5 Artifact and Threats

The review artifact includes the benchmark, annotation guide, blind-audit files, adjudication files, taxonomy repair logs, model predictions, evaluation tables, error appendix, training packet, and root-cause evidence extension. Public identity-bearing repository and archival links are intentionally omitted from the blinded paper; an anonymized artifact package can accompany the ATC submission.

The root-cause extension is not used to inflate the primary triage claim. It contains a 50-row human-adjudicated root-cause subset and a 250-row evidence-coded extension that records linked-fix signals, file categories, and evidence provenance. We include it because the extension supports the next systems question after triage: whether report symptoms can be connected to fixes in compiler, runtime, dtype, test, documentation, or kernel code.

Threats to validity remain. The benchmark is public-issue based and may not represent private industrial failures. The full 1,191-row benchmark is not double-adjudicated, although targeted audits and adjudication passes were used to repair label drift. Some reports lack minimal reproducers or confirmed fixes, so the task is report triage rather than definitive root-cause attribution. Cross-repository transfer remains weaker than shuffled evaluation and should be treated as an open systems problem.

6 Related Work

GPUTriage is related to three lines of work. First, GPU compiler and runtime systems such as Triton, TVM, PyTorch, CuPy, and Numba expose the failure surfaces studied in this paper. Second, software repository mining uses issue reports, bug reports, and pull requests as empirical software-engineering data; our contribution is a task-specific benchmark for GPU numerical-failure triage rather than a general issue-classification dataset. Third, LLMs and instruction-tuned sequence models are increasingly used for software tasks, but our evaluation emphasizes human label quality, weak-label failure, and cross-repository transfer rather than a single shuffled accuracy number.

7 Conclusion

GPUTriage turns noisy public GPU issue reports into a reusable benchmark and LLM-based triage system for numerical failures. Human labels improve strongly over weak search labels, a fine-tuned LLM reaches 80.5% held-out accuracy, and selective LLM/deterministic agreement reaches 89.3% accuracy at 68.3% coverage. The remaining weakness is cross-project transfer, where repository specific language and failure modes reduce accuracy to 66.8%. The result is a practical artifact for systems reliability research and a foundation for future root-cause prediction over GPU software failures.

Acknowledgments

Omitted for double-blind review.

References

[1] Tillet, P., Kung, H., and Cox, D. 2019. Triton: An intermediate language and compiler for tiled neural network computations. In *MAPL*.

[2] Okuta, R., Unno, Y., Nishino, D., Hido, S., and Loomis, C. 2017. CuPy: A NumPy-compatible library for NVIDIA GPU calculations. In *Workshop on Machine Learning Systems at NeurIPS*.

[3] Paszke, A. et al. 2019. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*.

[4] Lam, S. K., Pitrou, A., and Seibert, S. 2015. Numba: A LLVM-based Python JIT compiler. In *LLVM-HPC*.

[5] Chen, T. et al. 2018. TVM: An automated end-to-end optimizing compiler for deep learning. In *OSDI*.

[6] Raffel, C. et al. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR* 21, 140, 1–67.

[7] Chung, H. W. et al. 2024. Scaling instruction-finetuned language models. *JMLR* 25, 70, 1–53.